

TESTING_FRAMEWORK_SPECIFICATION

HelixTranslate Testing Framework Specification

Current Testing Status Analysis

Test Coverage by Package (Critical Priority)

Package	Current Coverage	Target	Gap	Priority
pkg/api	32.8%	100%	67.2%	CRITICAL
pkg/distributed	45.2%	100%	54.8%	CRITICAL
pkg/security	~60%	100%	40%	HIGH
pkg/websocket	~70%	100%	30%	HIGH
pkg/translator	~75%	100%	25%	MEDIUM
pkg/markdown	~80%	100%	20%	MEDIUM

Disabled Tests Analysis (61 Total)

Critical Disabled Tests

test/security/authentication_test.go - ENTIRE FILE DISABLED
test/security/input_validation_new_test.go - INPUT VALIDATION MISSING
test/integration/ssh_translation_test.go - SSH INFRASTRUCTURE MISSING
test/stress/translation_stress_test.go - PERFORMANCE TESTING DISABLED

Medium Priority Disabled Tests

test/performance/translation_performance_test.go - CONSISTENTLY SKIPPED
test/distributed/integration_test.go - DISTRIBUTED TESTING MISSING
test/e2e/translation_quality_e2e_test.go - E2E INFRASTRUCTURE

ISSUES

test/integration/batch_api_test.go – BATCH PROCESSING NOT TESTABLE

Testing Framework Implementation Plan

Phase 1: Test Infrastructure Restoration (Week 1)

1.1 WebSocket Testing Infrastructure

Current Issues:

- Hardcoded ports causing conflicts
- Race conditions in connection handling
- Missing mock implementations

Implementation Plan:

```
// New test infrastructure
type TestWebSocketHub struct {
    *websocket.Hub
    port      int
    server    *httptest.Server
    clients   map[string]*websocket.Conn
    messages  []WebSocketMessage
}

func NewTestWebSocketHub() *TestWebSocketHub {
    port := getFreePort()
    hub := websocket.NewHub()
    server := httptest.NewServer(http.HandlerFunc(func(w
        http.ResponseWriter, r *http.Request) {
            websocket.UpgradeConnection(hub, w, r)
        }))

    return &TestWebSocketHub{
        Hub:      hub,
        port:     port,
        server:   server,
        clients:  make(map[string]*websocket.Conn),
        messages: make([]WebSocketMessage, 0),
    }
}
```

1.2 SSH Testing Infrastructure

Current Issues:

- Non-existent test servers
- Incomplete authentication mocking
- Missing key pair management

Implementation Plan:

```
// Test SSH server implementation
type TestSSHServer struct {
    config      *ssh.ServerConfig
    listener    net.Listener
    clients     map[string]*ssh.ServerConn
    commands    []SSHCommand
    authFailed  bool
}

func NewTestSSHServer() (*TestSSHServer, error) {
    // Generate test key pair
    privateKey, err := rsa.GenerateKey(rand.Reader, 2048)
    if err != nil {
        return nil, err
    }

    // Create server config
    config := &ssh.ServerConfig{
        PasswordCallback: func(conn ssh.ConnMetadata, password
            []byte) (*ssh.Permissions, error) {
            if conn.User() == "testuser" && string(password) ==
                "testpass" {
                return nil, nil
            }
            return nil, fmt.Errorf("authentication failed")
        },
    }

    config.AddHostKey(privateKey)

    // Start listener
    listener, err := net.Listen("tcp", "127.0.0.1:0")
    if err != nil {
        return nil, err
    }

    return &TestSSHServer{
        config:    config,
        listener:  listener,
        clients:   make(map[string]*ssh.ServerConn),
        commands:  make([]SSHCommand, 0),
    }, nil
}
```

1.3 Security Test Infrastructure

Current Issues:

- HTTP server infrastructure missing
- Authentication mocking incomplete
- Input validation testing insufficient

Implementation Plan:

```
// Test HTTP server for security testing
type TestHTTPServer struct {
    server    *httptest.Server
    auth      *security.Authenticator
    rateLimit *security.RateLimiter
    logger    logger.Logger
}

func NewTestHTTPServer() *TestHTTPServer {
    auth := security.NewAuthenticator(security.AuthConfig{
        JWTSecret: "test-secret",
        EnableAuth: true,
    })

    rateLimit := security.NewRateLimiter(security.RateLimitConfig{
        RPS: 10,
        Burst: 20,
    })

    mux := http.NewServeMux()

    // Add protected routes
    mux.HandleFunc("/api/v1/translate",
        auth.Middleware(rateLimit.Middleware(handleTranslate)))
    mux.HandleFunc("/api/v1/auth", handleAuth)

    server := httptest.NewServer(mux)

    return &TestHTTPServer{
        server:    server,
        auth:      auth,
        rateLimit: rateLimit,
        logger:    logger.NewTestLogger(),
    }
}
```

Phase 2: Comprehensive Test Implementation (Week 2)

2.1 Test Coverage Enhancement Strategy

Critical Packages to 100% Coverage:

pkg/api - Complete Implementation

```
// Missing test coverage areas:
- All HTTP handlers (POST, PUT, DELETE)
- Error path testing
- Authentication middleware
- Rate limiting functionality
```

- File upload handling
- WebSocket upgrade process
- Request validation
- Response formatting

Implementation Example:

```
func TestTranslateHandler_CompleteCoverage(t *testing.T) {
    tests := []struct {
        name          string
        method        string
        path          string
        body          interface{}
        headers       map[string]string
        expectedStatus int
        expectedError string
        setupMock     func(*MockTranslator)
    }{
        {
            name: "Successful translation",
            method: "POST",
            path: "/api/v1/translate",
            body: map[string]interface{}{
                "input_file": "test.epub",
                "provider": "openai",
                "model": "gpt-4",
            },
            headers: map[string]string{
                "Content-Type": "application/json",
                "Authorization": "Bearer valid-token",
            },
            expectedStatus: http.StatusOK,
            setupMock: func(mt *MockTranslator) {
                mt.On("Translate", mock.Anything, mock.Anything,
                    mock.Anything).
                    Return("translated text", nil)
            },
        },
        {
            name: "Unauthorized access",
            method: "POST",
            path: "/api/v1/translate",
            body: map[string]interface{}{
                "input_file": "test.epub",
            },
            expectedStatus: http.StatusUnauthorized,
            setupMock: func(mt *MockTranslator) {
                // No mock setup - should not reach translator
            },
        },
    },
    // Add 20+ more test cases for complete coverage
}
```

```

    for _, tt := range tests {
        t.Run(tt.name, func(t *testing.T) {
            // Test implementation
        })
    }
}

```

pkg/distributed - Complete Implementation

// Missing test coverage areas:

- SSH pool management
- Worker coordination
- Version synchronization
- Performance optimization
- Failover mechanisms
- Load balancing
- Health monitoring
- Resource allocation

2.2 Integration Testing Framework

Cross-Package Integration Tests:

```

func TestCompleteTranslationWorkflow_Integration(t *testing.T) {
    // Setup complete system
    eventBus := events.NewEventBus()
    translator := translator.NewLLMTranslator(config)
    websocketHub := websocket.NewHub()
    apiServer := api.NewServer(translator, eventBus, websocketHub)

    // Test complete workflow
    testFile := createTestEpub(t, "Test Book", "Test content")

    // Start servers
    go websocketHub.Run()
    go apiServer.Start(":0")

    // Simulate API request
    resp := makeTranslationRequest(apiServer.Addr(), testFile)

    // Verify complete workflow
    assert.Equal(t, http.StatusOK, resp.StatusCode)

    // Verify events were emitted
    events := eventBus.GetEvents()
    assert.Contains(t, events, events.EventTranslationStarted)
    assert.Contains(t, events, events.EventTranslationProgress)
    assert.Contains(t, events, events.EventTranslationCompleted)
}

```

2.3 Performance Testing Framework

Load Testing Implementation:

```
func TestTranslationAPI_Performance(t *testing.T) {
    if testing.Short() {
        t.Skip("Skipping performance test in short mode")
    }

    server := setupTestServer(t)
    defer server.Close()

    // Performance test parameters
    concurrency := 100
    requests := 1000
    duration := 30 * time.Second

    // Create test data
    testFiles := createTestFiles(t, 10)

    // Run load test
    var wg sync.WaitGroup
    results := make(chan TestResult, requests)

    for i := 0; i < concurrency; i++ {
        wg.Add(1)
        go func(workerID int) {
            defer wg.Done()

            for j := 0; j < requests/concurrency; j++ {
                start := time.Now()

                file := testFiles[j%len(testFiles)]
                resp := makeTranslationRequest(server.URL, file)

                duration := time.Since(start)
                results <- TestResult{
                    WorkerID:    workerID,
                    RequestID:     j,
                    Duration:      duration,
                    StatusCode:    resp.StatusCode,
                    Success:       resp.StatusCode == http.StatusOK,
                }
            }
        }(i)
    }

    wg.Wait()
    close(results)

    // Analyze results
    analyzePerformanceResults(t, results)
}
```

Phase 3: Advanced Testing Features (Week 3)

3.1 Mutation Testing Framework

Implementation:

```
func TestWithMutations(t *testing.T) {
    // Define mutations
    mutations := []Mutation{
        {
            Type: "arithmetic_operator",
            From: "+",
            To:   "-",
        },
        {
            Type: "logical_operator",
            From: "&&",
            To:   "||",
        },
        {
            Type: "conditional_boundary",
            From: ">",
            To:   ">=",
        },
    },
}

for _, mutation := range mutations {
    t.Run(fmt.Sprintf("mutation_%s", mutation.Type), func(t
    *testing.T) {
        // Apply mutation
        original := applyMutation(mutation)

        // Run tests
        failed := runTestsWithMutation(t, mutation)

        // Verify mutation was caught
        assert.True(t, failed, "Mutation should have been
        caught: %v", mutation)

        // Restore original
        restoreCode(original)
    })
}
```

3.2 Fuzz Testing Integration

Implementation:

```
func FuzzTranslatorAPI(f *testing.F) {
    // Seed corpus
    f.Add([]byte("test content"))
}
```



```

f.Add([]byte("special chars: !@#$%^&*()"))
f.Add([]byte("unicode: 你好世界 🌐"))
f.Add([]byte("very long content " + strings.Repeat("x",
    10000)))

f.Fuzz(func(t *testing.T, input []byte) {
    // Test translation with fuzzed input
    translator := setupTestTranslator(t)

    _, err := translator.Translate(context.Background(),
        string(input), "test context")

    // Verify no panics or crashes
    assert.NotPanics(t, func() {
        // Translation should handle any input gracefully
    })

    // Verify error handling
    if err != nil {
        assert.IsType(t, &translator.TranslationError{}, err)
    }
})
}

```

3.3 Property-Based Testing

Implementation:

```

func TestTranslationProperties(t *testing.T) {
    property := properties.NewPropertyTest(t)

    property.Property("translation should preserve structure",
        func(input TestBook) bool {
            translator := setupTestTranslator(t)

            result, err := translator.TranslateBook(input)
            if err != nil {
                return false
            }

            // Verify chapter count preserved
            return len(result.Chapters) == len(input.Chapters)
        })

    property.Property("translation should be reversible for
        simple text", func(text string) bool {
        if len(text) > 1000 {
            return true // Skip very long texts
        }

        translator := setupMockTranslator(t)

```

```

// Translate forward
forward, err :=
    translator.Translate(context.Background(), text, "en->sr")
if err != nil {
    return false
}

// Translate backward
backward, err :=
    translator.Translate(context.Background(), forward, "sr-
    >en")
if err != nil {
    return false
}

// Should be approximately equal (allowing for translation
// differences)
return similarity(text, backward) > 0.8
})
}

```

Test Infrastructure Components

Test Utilities Package (test/utls/)

```

// File creation utilities
func CreateTestEpub(t *testing.T, title, content string) string
func CreateTestFb2(t *testing.T, title, content string) string
func CreateTestPdf(t *testing.T, title, content string) string
func CreateTestMarkdown(t *testing.T, content string) string

// Server utilities
func StartTestAPIServer(t *testing.T, translator
    translator.Translator) *httptest.Server
func StartTestWebSocketServer(t *testing.T) (*websocket.Hub,
    *httptest.Server)
func StartTestSSHServer(t *testing.T) *TestSSHServer

// Mock utilities
func CreateMockTranslator(t *testing.T) *MockTranslator
func CreateMockLLMClient(t *testing.T, provider string)
    *MockLLMClient
func CreateMockEventBus(t *testing.T) *MockEventBus

// Database utilities
func SetupTestDatabase(t *testing.T) *sql.DB
func SetupTestRedis(t *testing.T) *redis.Client
func CleanupTestDatabase(t *testing.T, db *sql.DB)

```

Test Fixtures (test/fixtures/)

```
test/fixtures/
├── ebooks/
│   ├── small_book.fb2
│   ├── medium_book.epub
│   ├── large_book.pdf
│   └── special_chars.md
├── configs/
│   ├── openai_config.json
│   ├── ssh_config.json
│   └── distributed_config.json
├── translations/
│   ├── fb2_expected_sr.json
│   ├── epub_expected_fr.json
│   └── markdown_expected_de.json
└── data/
    ├── sample_requests.json
    ├── error_responses.json
    └── performance_data.json
```

Automated Testing Pipeline

Continuous Integration Testing

```
# .github/workflows/comprehensive-testing.yml
name: Comprehensive Testing

on:
  push:
    branches: [ main, develop ]
  pull_request:
    branches: [ main ]

jobs:
  unit-tests:
    runs-on: ubuntu-latest
    steps:
      - uses: actions/checkout@v3
      - uses: actions/setup-go@v3
        with:
          go-version: '1.25.2'

      - name: Install dependencies
        run: go mod download

      - name: Run unit tests
        run: go test -v -race -coverprofile=coverage.out ./pkg/...

      - name: Upload coverage
```

```
    uses: codecov/codecov-action@v3
    with:
      file: ./coverage.out

integration-tests:
  runs-on: ubuntu-latest
  services:
    postgres:
      image: postgres:15
      env:
        POSTGRES_PASSWORD: test
    redis:
      image: redis:7

  steps:
    - uses: actions/checkout@v3
    - uses: actions/setup-go@v3

    - name: Run integration tests
      run: go test -v -tags=integration ./test/integration/...

performance-tests:
  runs-on: ubuntu-latest
  if: github.event_name == 'push' && github.ref == 'refs/heads/main'

  steps:
    - uses: actions/checkout@v3
    - uses: actions/setup-go@v3

    - name: Run performance tests
      run: go test -v -timeout=30m ./test/performance/...

security-tests:
  runs-on: ubuntu-latest
  steps:
    - uses: actions/checkout@v3
    - uses: actions/setup-go@v3

    - name: Run security tests
      run: go test -v ./test/security/...

    - name: Run security scan
      uses: securecodewarrior/github-action-add-sarif@v1
      with:
        sarif-file: 'security-scan-results.sarif'
```

Test Coverage Requirements

Minimum Coverage Standards

Package Type	Minimum Coverage	Target Coverage
Core Packages (api, distributed, security)	95%	100%
Utility Packages (logger, hash, events)	90%	100%
Feature Packages (translator, markdown, preparation)	85%	100%
Test Utilities	80%	100%
Mock Implementations	75%	100%

Coverage Reporting Tools

Generate comprehensive coverage report

```
make test-coverage
```

Generate HTML coverage visualization

```
go tool cover -html=coverage.out -o coverage.html
```

Generate coverage by package

```
go test -coverprofile=pkg_api.out ./pkg/api/...
```

```
go test -coverprofile=pkg_distributed.out ./pkg/distributed/...
```

Mutation testing

```
go install github.com/zimmski/oss-mutate@latest
```

```
oss-mutate -format=json -output=mutation-report.json ./...
```

This comprehensive testing framework specification provides the foundation for achieving 100% test coverage and production-quality testing across the entire HelixTranslate project.